WEEK 26 –23 ·3 NOVEMBER 2026

GeoAI methods for hyperspectral applications

Deep learning on spectra and patches — what it adds, and where it breaks

Week 3 · Session 1 · practical: 20_geoai_deep_learning.ipynb

LEARNING OBJECTIVES

What you will be able to do

- Choose between pixel, patch and scene models for a hyperspectral task
- Explain how 1D, 2D and 3D convolutions use spectral and spatial information
- Say what a **loss**, a **gradient step** and an **epoch** actually are, and read a training curve
- Set up training that does not fool you: blocked splits, buffers, class weights, early stopping
- Describe what foundation models and self-supervised pre-training offer, and their limits
- Recognise **domain shift** and quantify it before deploying a model elsewhere

1. What GeoAI adds

Three ways to look at a hyperspectral cube

THE CORE IDEA

Classical models see a list of numbers. Deep models can see a neighbourhood.

A random forest treats each pixel's 352 bands as an unordered feature vector. A convolutional network knows that band 120 sits next to band 121, and that this pixel sits next to that one.

Three model families

Family	Input	Typical architectures	Main weakness
Pixel / spectral	one spectrum (~350 values)	RF, SVM, 1D-CNN, SpectralFormer (Hong et al. 2022)	no spatial context, salt-and-pepper maps
Patch / spectral–spatial	small cube, e.g. 9 × 9 × bands	2D-CNN on PCA, 3D-CNN (Chen et al. 2016), HybridSN (Roy et al. 2020)	leakage risk, more compute, blurred boundaries
Scene / segmentation	a whole tile, e.g. 128 × 128	Fully convolutional nets, U-Net, segmentation transformers	needs *dense* labels — rare in hyperspectral

2. How a network actually learns

Weights, loss, one small step — repeated

A layer is weights and a non-linearity

- A **layer** multiplies its inputs by a table of numbers (**weights**), adds an offset, then passes the result through a simple non-linear function
- Stack a few layers and the network can represent shapes a single band ratio cannot
- **Training = choosing the weights.** Nothing else about the model changes
- You already use a fixed-weight model: NDVI is (+1, −1) on two bands, chosen by a human in 1973
  - Our 1D-CNN holds **42,309** such numbers; HybridSN holds **265,349**

The loss: one number for how wrong the model is

- Show the network a **batch** of pixels, let it guess, and score the guesses with a **loss function**
- For classification we use **cross-entropy**: it is small when the right class gets a high probability, and large when the model is confidently wrong
- Our loss is **class-weighted**, so a mistake on cropland (2,166 training pixels) costs more than one on mangrove (33,995)
- The loss is what the optimiser minimises — but it is *not* the number you report
  - Judge the model on **macro-F1** over a blocked test set; watch the loss only to see if training is working

One gradient step, and the learning rate

- Compute the loss on one batch, then ask: **which way should each weight move to make it smaller?**
- That direction is the **gradient**; computing it backwards through the layers is *backpropagation*
- Nudge every weight a little in that direction — one **gradient step**. That is the whole algorithm
- The **optimiser** (we use Adam) decides the step; the **learning rate** (1e-3 here) sets its size
- Too large and the loss jumps around; too small and it crawls. It is the first thing to adjust

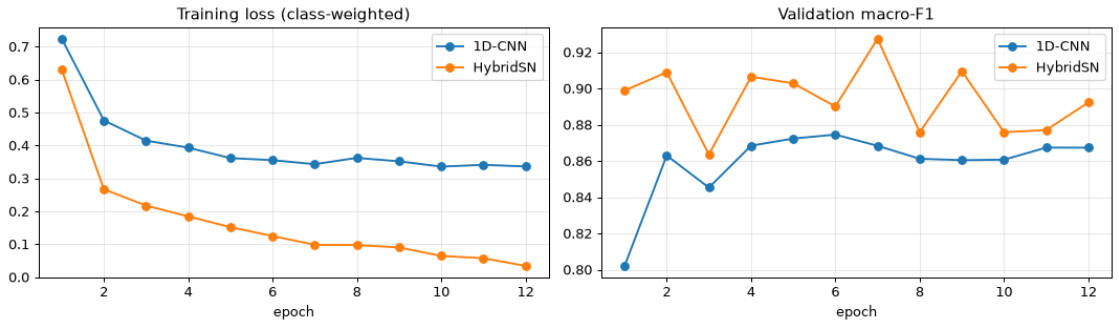
Epoch, batch and iteration — in this notebook's numbers

Term	What it means	In notebook 20
Batch	the group of pixels scored before one weight update	128 pixels
Iteration	one batch in, one gradient step out	~57 per epoch
Epoch	one pass through all the training data	7,332 training pixels
Training run	epochs until it stops improving	stopped at 12 of a possible 40
Total updates	iterations × epochs	~690 gradient steps

Dropout, batch normalisation, weight decay

- **Dropout** switches off a random share of units each batch (0.3 and 0.4 in our models), so the network cannot lean on any single feature — it is the main brake on overfitting
- **Batch normalisation** rescales each layer's outputs to a stable range, which makes training faster and far less sensitive to the learning rate
- **Weight decay** (1e-4) gently pulls weights towards zero, preferring simpler solutions
- All three fight the same enemy: a model that memorises 7,332 pixels instead of learning spectra
  - Later, dropout earns a second job: left on at prediction time it gives the MC-dropout uncertainty you will map in section 5

Reading the training curves



Both models, 12 epochs, class-weighted loss

- Left: **training** loss. Right: **validation** macro-F1 — the one that matters
- HybridSN's loss falls to 0.03 while its validation score stops improving: that gap is **overfitting**
- **Early stopping** keeps the best epoch (7, macro-F1 0.927) and gives up after a few more
- Safe to change: learning rate, epochs, patch size, number of PCA components
- Leave alone until the curves make sense: layer counts, kernel sizes, the optimiser

3. The families in detail

What each architecture does with a hyperspectral cube

1D-CNN: convolving along the spectrum

- Kernels slide along the **wavelength axis**, learning shapes: slopes, peaks, absorption dips
- Equivalent to learning the band-maths that indices hard-code, but fitted to your data
- Small and fast: our model has **42,309 parameters** and trains in about 40 seconds on a laptop CPU
- Applies to any pixel of any scene — no neighbourhood needed, so no patch-leakage risk
  - Input must be standardised per band, or the first layer is dominated by the brightest region

3D and hybrid networks: adding the neighbourhood

- A **3D convolution** moves a kernel through rows, columns *and* wavelengths at once
- **HybridSN** (Roy et al. 2020): a few 3D layers for spectral–spatial features, then 2D layers for texture
- Almost always run on **PCA-reduced** bands — 30 components here explain 99.99 % of the variance
- Our patch model has **265,349 parameters**, six times the 1D-CNN, and still trains in ~35 s
- Smoother, more accurate maps — at the cost of blurred class boundaries and mixed-pixel errors

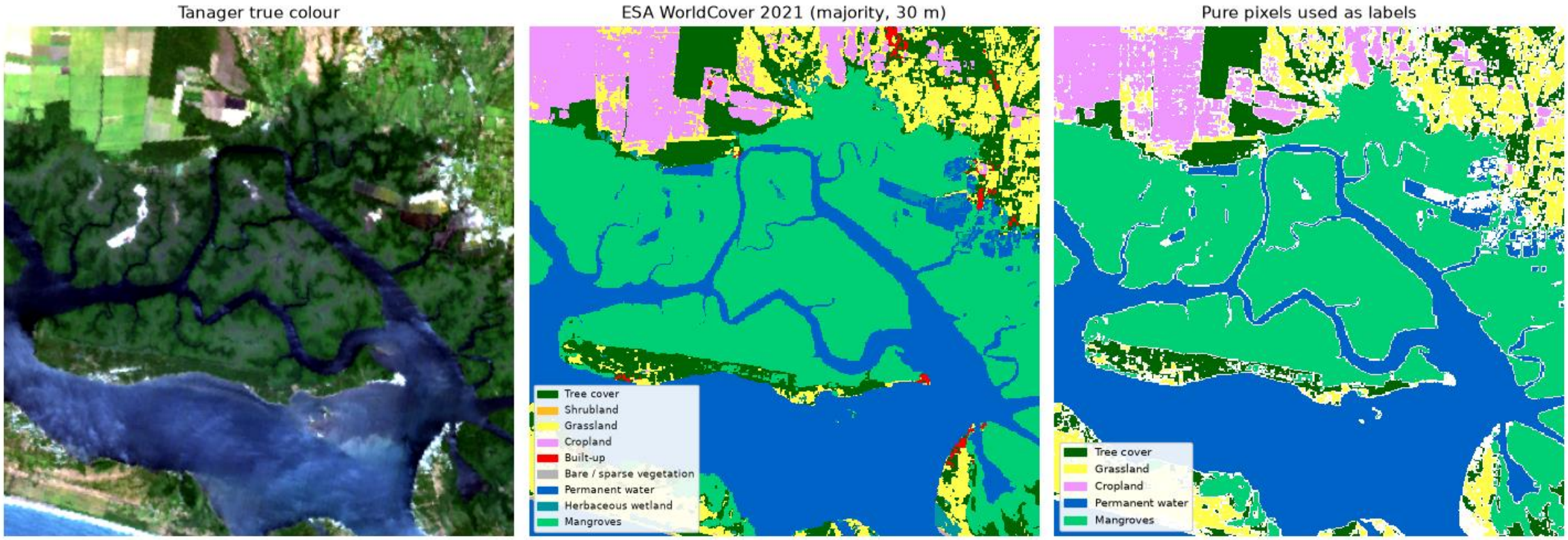
Attention and transformers

- Self-attention lets every band (or patch) weigh every other, instead of only its neighbours
- **SpectralFormer** (Hong et al. 2022) treats groups of adjacent bands as tokens
- Strong when relationships are **long-range** — a feature at 700 nm modulating one at 2200 nm
- Data-hungry: with a few thousand labelled pixels, a CNN or even a random forest often wins
  - Rule of thumb: try the simple model first and make the complex one prove its worth

4. Making it work

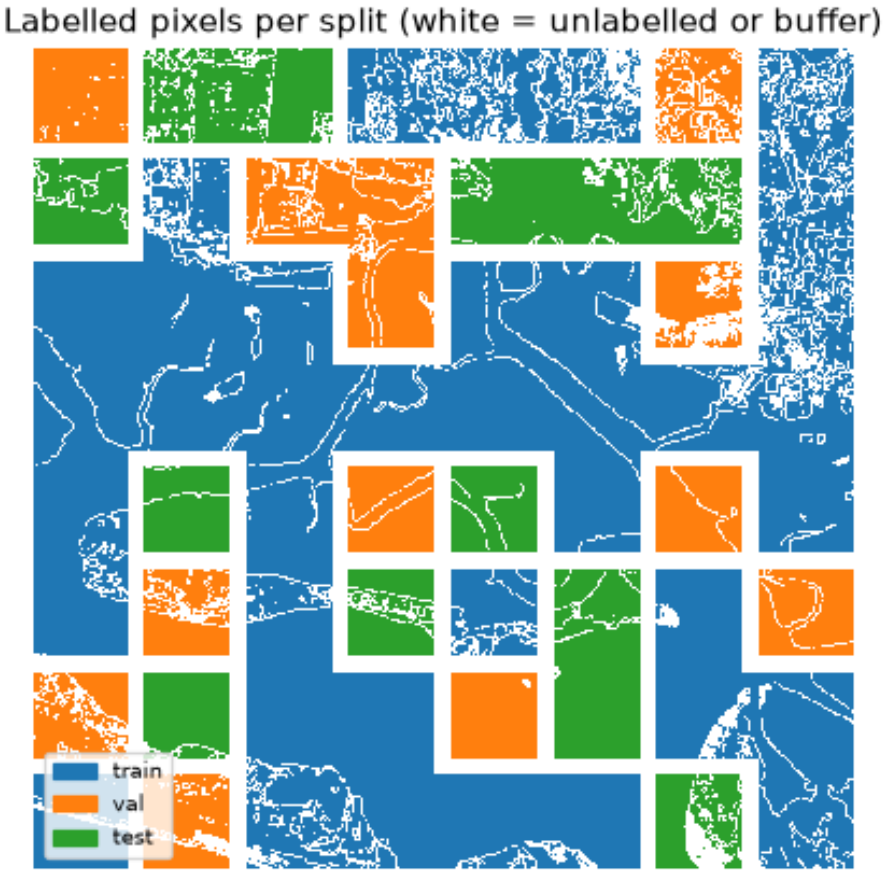
The parts that decide whether your number is real

The data: a Tanager window and its labels



El Salvador, 400 × 400 px, 352 good bands, five WorldCover classes (pure pixels only)

Split by blocks, and buffer the patches



Spatial blocks with a buffer for patch models

- Blocks of 50 px (1.5 km) assigned to train, validation and test
- A **9 × 9 patch** centred near a block edge reaches into the next block
- So we also drop pixels whose full patch is not inside one split — the **buffer**
- Without it, test patches literally contain training pixels
- This is the single most common way to publish an inflated accuracy

Training essentials, in order of how often they are forgotten

- **Standardise** inputs per band, using statistics from the *training* set only
- **Class weights** or balanced sampling — our classes range from 9,993 to 57,479 pure pixels
- **Early stopping** on a validation set: both our models stopped after 12 epochs
- **Fix every seed** (numpy, torch) and record it, or your results are not reproducible
- Monitor **macro-F1**, not loss, and not overall accuracy

Label scarcity is the real constraint

- Our training set is 7,332 pixels — large for hyperspectral, tiny for deep learning
- Labels are **weak**: ESA WorldCover 2021 versus Tanager imagery from 2025
- Deep models with 10^5–10^6 parameters overfit such sets easily
- Mitigations: augmentation, class weighting, early stopping, smaller models — and pre-training
- **Always report a random forest next to your deep model**

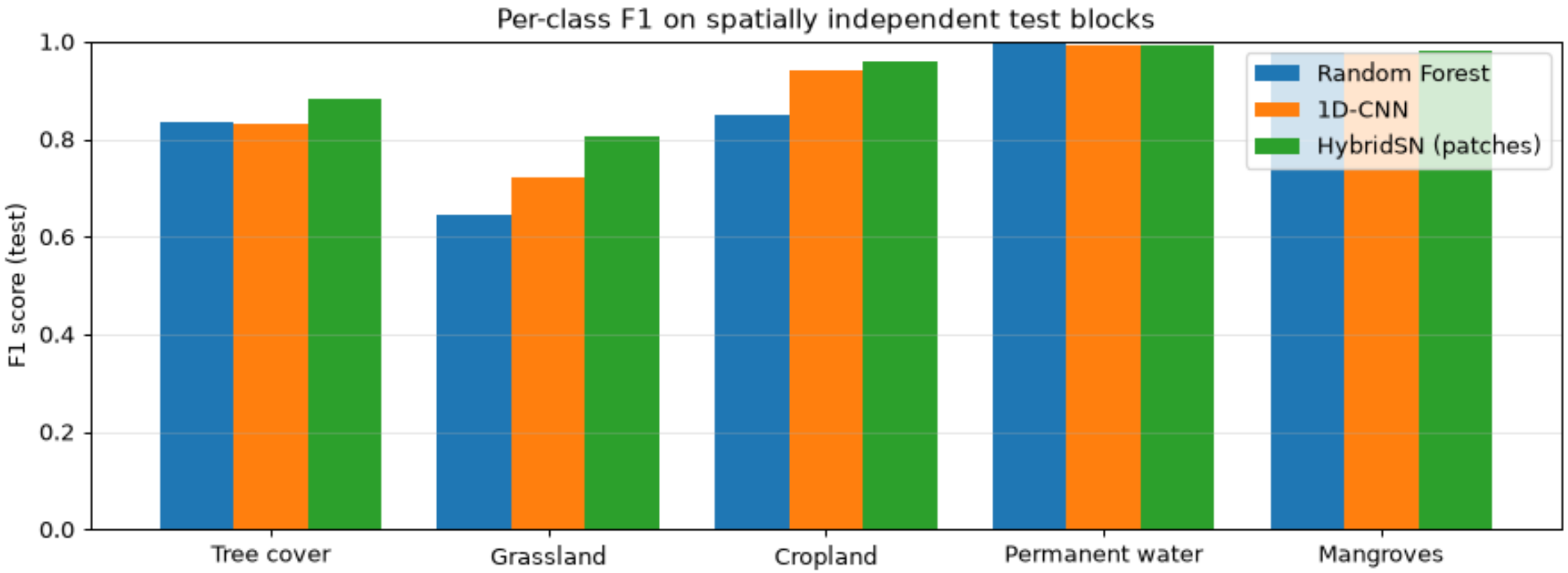
5. What the models actually did

Measured on spatially separated test blocks

Three models, one window, blocked test set

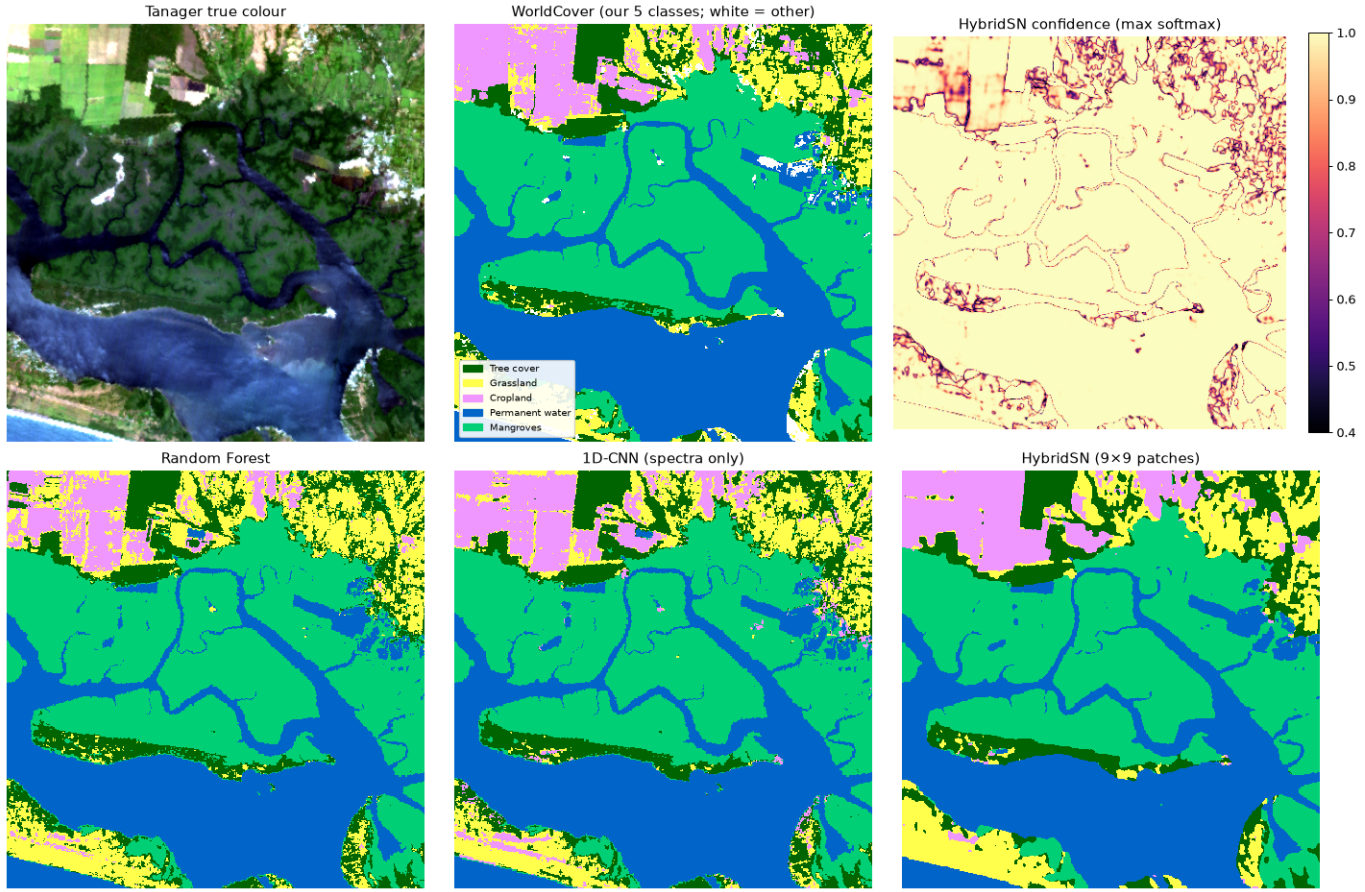
Model	Parameters	Training time	Overall accuracy	Macro F1
Random forest (baseline)	—	2.5 s	0.925	0.861
1D-CNN on spectra	42,309	41 s	0.940	0.892
HybridSN, 9 × 9 patches	265,349	35 s	**0.959**	**0.925**

Where the gain actually came from



Per-class F1 on the test blocks: random forest, 1D-CNN, HybridSN

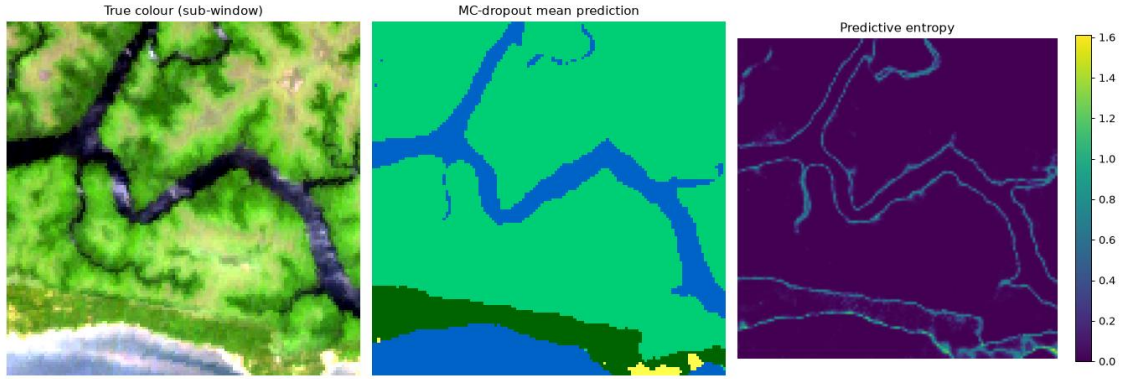
From model to map



True colour, reference labels, confidence, and the three model maps

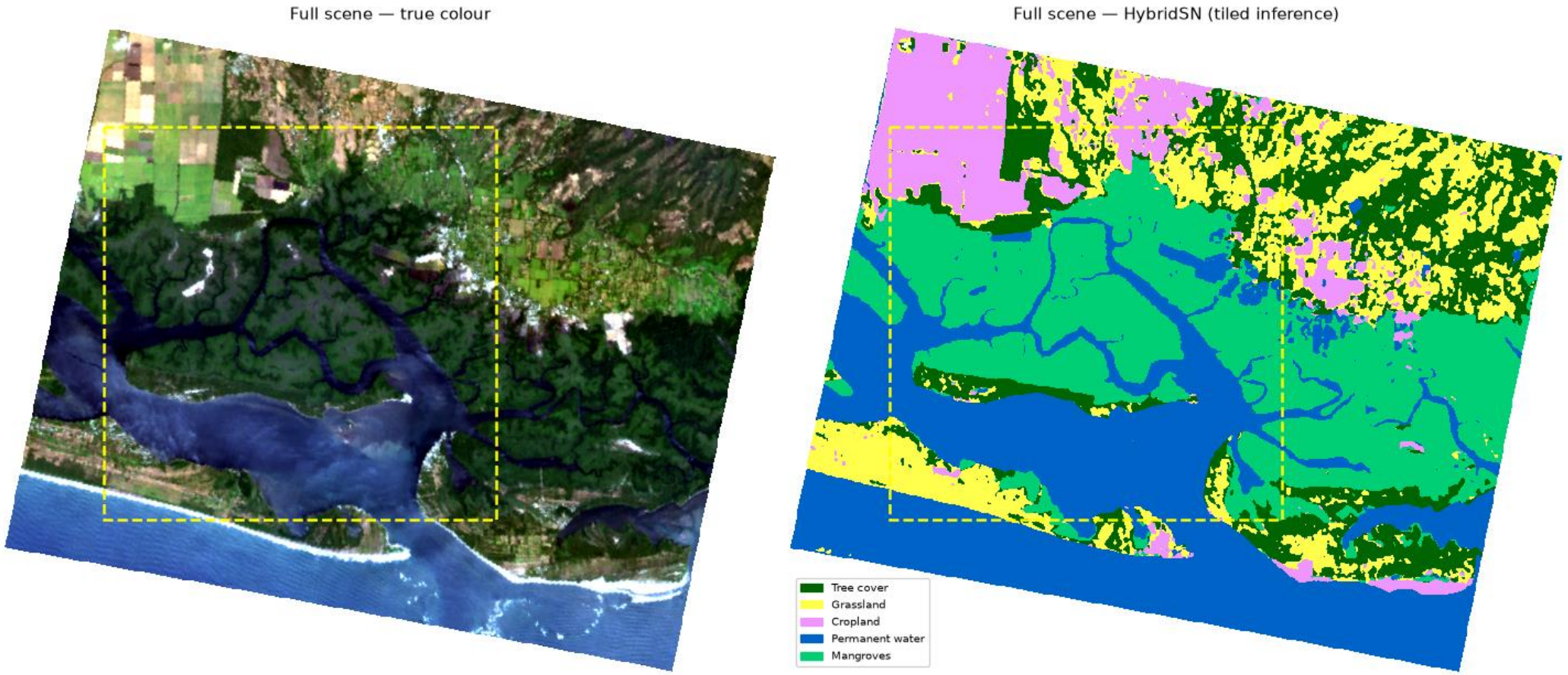
Confidence and uncertainty

- Max softmax is a **confidence score**, not a calibrated probability
- **MC dropout**: run the network several times with dropout active and look at the spread
- Ten passes over 16,900 pixels took 32 s
- High uncertainty concentrates at class boundaries and mixed pixels
- Ship the uncertainty layer; let the user decide the threshold



Monte-Carlo dropout uncertainty on a sub-window

Scaling to the whole scene



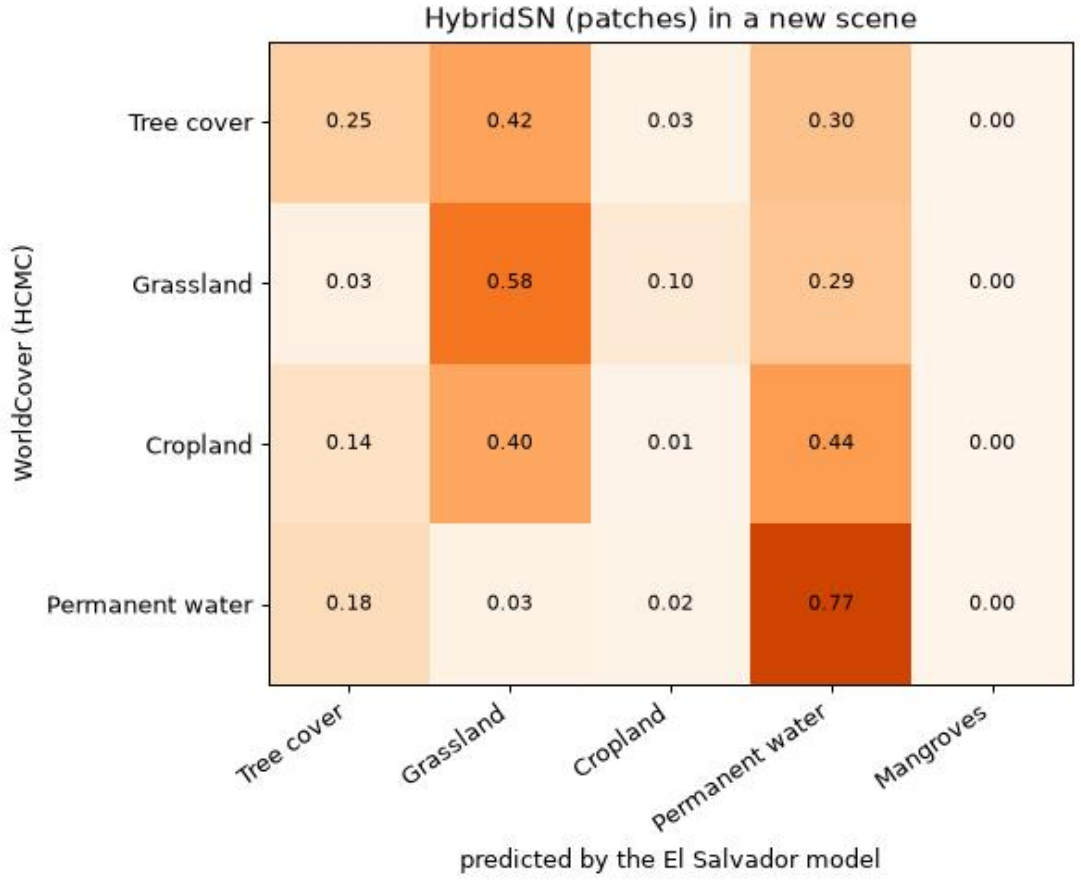
Tiled sliding-window inference over 655 × 785 pixels, 88 s on CPU

6. Where it breaks

Domain shift, and the limits of a trained model

Does the model travel?





The El Salvador model applied to Ho Chi Minh City

The same models, three test sets

Test set	Random forest	1D-CNN	HybridSN
El Salvador, blocked test	0.925	0.940	0.959
Rest of the same scene	—	—	0.843
Ho Chi Minh City	0.261	0.334	0.335
…of which predicted 'mangrove'	41 %	44 %	0 %

DOMAIN SHIFT, IN ONE NUMBER

The random forest labelled 41 % of Ho Chi Minh City as mangrove.

Different species, background, atmosphere, illumination and class definitions. The model was never wrong about El Salvador — it was simply asked a question outside its experience.

Living with domain shift

- Validate in the domain you will deploy in — a new site, a new date, a new sensor
- Collect a small number of **local labels**; even a few hundred help a great deal
- Consider domain adaptation, fine-tuning, or simply retraining per scene
- Use the **confidence layer** as an early warning: off-domain data often looks uncertain
- State the validated domain in your metadata, as notebook 31's product does

7. Foundation models

Using unlabelled data to fix the label bottleneck

Self-supervised pre-training, in one slide

- Pre-train on a **large unlabelled archive** with a pretext task — masked reconstruction, or matching two views of the same place
- The model learns general spectral–spatial structure without any labels
- Then **fine-tune** on your few thousand labelled pixels, or train a small classifier on its embeddings
- The promise: far fewer labels for the same accuracy
  - The field moves fast — check current versions and benchmarks before committing

READ THE FINE PRINT

Models you will meet in the literature

Model	Pre-trained on	Note for us
HyperSIGMA (Wang et al. 2024/25)	Hyperion and GaoFen-5 hyperspectral	genuinely hyperspectral
SpectralEarth (Braham et al. 2024/25)	EnMAP patches	closest sensor to Tanager
SpectralGPT (Hong et al. 2024)	Sentinel-2 multispectral	despite the name, not hyperspectral
Prithvi (Jakubik et al. 2023)	Harmonized Landsat–Sentinel-2, 6 bands	multispectral
DOFA (Xiong et al. 2024)	multiple sensors, wavelength-aware	can ingest varied band sets

Sensor mismatch: the practical catch

- A model pre-trained on Sentinel-2 or EnMAP has **never seen Tanager's 426-band grid**
- Either resample Tanager to the model's bands (`hu.resample_to_sensor`, notebook 30)
- …or use a **wavelength-aware** architecture that conditions on each band's centre
- At the time of writing, few if any public foundation models are trained on Tanager itself
- Resampling throws away exactly the narrow-band detail you bought the sensor for

Compute budget: what runs where

Task	Laptop CPU	Single GPU
Random forest, 7k pixels	2.5 s	not needed
1D-CNN, 12 epochs	41 s	seconds
HybridSN, 12 epochs	35 s	seconds
Full-scene tiled inference	88 s	~10 s
Fine-tuning a foundation model	impractical	hours

PRACTICAL

Over to you

▶ `20_geoai_deep_learning.ipynb` · `120 min`

- Build the blocked split with a buffer and check the class counts
- Train the random forest baseline, then the 1D-CNN, then HybridSN
- Compare per-class F1 and explain which classes gained and why
- Map the full scene with tiled inference and inspect the confidence layer
- Apply your model to Ho Chi Minh City and quantify the domain shift

KEY TAKEAWAYS

Recap

- Pixel, patch or scene: choose by **how much context** the class needs and how many labels you have
- Deep learning gained macro-F1 0.861 → 0.925 here, mostly on the **hard** classes
- **Blocked splits with buffers** are what make that number believable
- Accuracy collapsed off-domain: 0.96 → 0.26 in a new city, 41 % of it labelled mangrove
- Foundation models address the label bottleneck, but mind the **sensor mismatch**

Next: Session 3 — Validation methods and accuracy assessment (notebook 21)

References and further reading

Chen, Y. et al. (2016). Deep feature extraction and classification of hyperspectral images based on CNNs. IEEE TGRS 54, 6232–6251.

Roy, S.K. et al. (2020). HybridSN: exploring 3D–2D CNN feature hierarchy for hyperspectral image classification. IEEE GRSL 17, 277–281.

Hong, D. et al. (2022). SpectralFormer: rethinking hyperspectral image classification with transformers. IEEE TGRS 60, 1–15.

Roberts, D.R. et al. (2017). Cross-validation strategies for data with spatial structure. Ecography 40, 913–929.

Ploton, P. et al. (2020). Spatial validation reveals poor predictive performance of large-scale ecological mapping models. Nature Comms 11, 4540.

Guo, C. et al. (2017). On calibration of modern neural networks. ICML.

Wang, D. et al. (2024/25). HyperSIGMA; Braham, N.A.A. et al. (2024/25). SpectralEarth; Hong, D. et al. (2024). SpectralGPT; Jakubik, J. et al. (2023). Prithvi; Xiong, Z. et al. (2024). DOFA.

Course material: notebook 20, quiz quizzes/quiz_w3_geoai_validation.md